Operationalizing Agent Provenance: A Structural Analysis of the Stela Pointer-First Constitution and Context Hazard Protocols

1. Executive Summary: The Governance of Stateless Intelligence

The Stela repository represents a paradigm shift in the architecture of AI-assisted software development, moving away from ephemeral, prompt-based interactions toward a rigorous, provenance-backed state machine. In this architecture, the repository is not merely a passive storage medium for code; it is the active cognitive architecture of the system.[1] The Policy of Truth (PoT) serves as the immutable constitution, dictating that the documentation is not a description of the system, but the law by which the system functions. This "stateless" philosophy—where "Context is not memory; it is a file"—is designed to eliminate the greatest risk in AI engineering: Drift.[1]

However, a critical asymmetry currently exists within Stela's operational fabric. While the Skills System (docs/library/skills/) has achieved a state of high maturity through the implementation of automated "Harvest and Promote" workflows—ensuring every skill has cryptographic provenance and is treated as a distinct operational artifact—the Agent System (docs/library/agents/) remains structurally immature. Agents are currently defined as static markdown files (e.g., R-AGENT-01) that often duplicate behavioral instructions found in the PoT, violating the "Pointer-first" doctrine.[1] This static nature renders them susceptible to obsolescence and hallucination, as they lack the verifiable link to successful execution that characterizes the Skills System.

This report provides an exhaustive architectural analysis and operational specification for updating the Agent System to conform to Stela's operational specifications. By adapting the Harvest/Promote logic currently used for skills, Stela can transition agents from static role definitions into dynamic, provenance-backed operational assets. Furthermore, this analysis defines the mechanisms for Context Hazard enforcement and Seclusion, ensuring that agents operate in hermetically sealed environments that prevent context contamination and align with the Filing Doctrine.[1]

1.1 The Stela State Machine

Stela functions as a text-based state machine where the file system constitutes the memory,

logic, and law of the operation.[1] Unlike traditional development environments where "context" is a vague accumulation of tribal knowledge, Slack history, and implicit assumptions, Stela operates on the axiom that if an instruction, rule, or fact is not present in the loaded context files (the Manifest), it does not exist for the purpose of the current session.[1]

This axiom necessitates a rigorous mechanism for "Context Injection." The system must strictly control what enters the AI's limited context window to prevent "Context Hazard"—a state where the AI is overwhelmed by irrelevant or contradictory information, leading to hallucinations or non-compliant behavior.[1] The primary mechanism for this control is the ops/kernel, specifically the ops/bin/llms and ops/bin/dump binaries, which act as the gatekeepers of the system's cognitive reality.

1.2 The Filing Doctrine and Jurisdictional Physics

The Filing Doctrine, defined in PoT.md Section 1.1, separates the repository into four distinct jurisdictions, each governed by specific physics and intent. This separation is crucial for the proposed "Seclusion" architecture, as it defines the boundaries of what an agent can "see" and "touch".[1]

Jurisdiction	Path	Function	Physics
Run	ops/	Executable Kernel	Contains binaries, manifests, and scripts. This is the "enforcement arm" of the constitution. Logic here must be deterministic and is the only place runtime logic belongs.
Explain	docs/	Legislative Branch	Contains manuals, rationale, and the PoT itself. It explains *why* the system functions. Agents and Skills live here as "Library" artifacts.

Work	projects/	Payload Sector	Where value is created. Projects are consumers of platform governance but must never define governance that contradicts the PoT.
Trash	storage/	Ephemeral Zone	Local artifacts, dumps, and handoff files. These are never canon and are excluded from the repository's permanent record via .gitignore.

The "Context Hazard" referenced in the current architectural crisis stems from a violation of this doctrine—specifically, when artifacts from the "Explain" layer (like the entire library of skills or agents) are forced into the "Run" layer's active context without filtration, overwhelming the AI's cognitive window.[1]

1.3 Staffing Protocol and Authority Hierarchy

The Staffing Protocol (PoT.md Section 4.1) defines a rigid hierarchy of authority that must be encoded into the new Agent definitions. This hierarchy is not merely organizational; it is cryptographic and functional.[1]

- **Operator (Human):** Holds absolute sovereignty. Only the Operator can commit, push, or merge. The Operator owns "secrets" and final approvals. The Operator is the biological validation layer.
- **Integrator (Lead AI):** The guardian of the PoT. The Integrator's role is not to write code, but to maintain "structural integrity" and detect "Drift." The Integrator generates Dispatch Packets (DPs)—immutable contracts for work. The Integrator is the State Manager.
- **Contractor (Guest AI):** The executor. Contractors are instantiated to perform specific tasks defined in a DP. They act as "Guest AIs" and must adhere to strict behavioral logic (no contractions, absolute literalism). The Agent definitions (R-AGENT-XX) are essentially Role Cards for these Contractors.[1]

The upgrade to the Agent System aims to formalize the **Contractor** role by replacing static role cards with provenance-backed personas harvested from successful work sessions.

---

2. Architectural Asymmetry: The Agent vs. Skill Gap

To understand the necessity of the proposed upgrade, we must analyze the current disparity between the Skills System and the Agent System. The Skills System serves as the "Gold Standard" for provenance-backed AI operations, while the Agent System remains a legacy artifact susceptible to drift.[1]

2.1 The Skills System: A Model of Provenance

The Skills System (docs/library/skills/) has successfully operationalized the concept of "Harvest and Promote," solving the problem of how to capture ephemeral AI workflows into durable assets.[1] By dissecting the ops/lib/scripts/skill.sh script provided in the repo dump, we can identify the mechanisms that make this system robust.[1]

2.1.1 Forensic Analysis of the Harvest Engine

The skill.sh script does not rely on human memory to write documentation; it relies on forensic capture. When a Contractor successfully completes a task, the Operator invokes ops/lib/scripts/skill.sh harvest. This command triggers a provenance capture sequence defined in ops/lib/scripts/heuristics.sh.[1]

- **DP-ID Capture:** The script extracts the active Dispatch Packet ID from TASK.md, linking the skill to a specific authorized work order. This ensures that no skill enters the library without a "work order" origin.
- **Git State Forensics:** It captures the HEAD commit hash and the branch name. This provides a cryptographic link to the exact state of the codebase when the skill was validated.
- **Friction Context Analysis:**
    - **Hot Zone Detection:** The script uses git diff --numstat to identify which files changed the most during the session. If the Contractor spent 90% of its effort in ops/bin/dump, the script automatically tags the "Hot Zone" as ops/bin/dump.
    - **High Churn Detection:** It analyzes git log to find files that were touched frequently, indicating instability or complexity ("High Churn").[1]

This data is written into a YAML-like "Provenance" header in the draft markdown file. This block is immutable; it proves that the skill was not "hallucinated" by an LLM in a vacuum but was born from actual, verified work.

2.1.2 The Semantic Collision Check

The script includes a check_semantic_collision function. It scans existing skills in docs/library/skills/ and current drafts in storage/handoff/ for keyword overlaps. This prevents "Library Bloat" drift, where multiple slightly different versions of "How to Refactor" clog the context.[1] This logic is critical for the Agent System to prevent duplicate personas (e.g., "Code Cleaner" vs. "Refactor Specialist").

2.2 The Agent System: The "Static" Flaw

Currently, the Agent System (docs/library/agents/) operates on an older, static model that predates the rigor of the Skills System. Examination of the R-AGENT files in the dump reveals significant structural weaknesses that contribute to system drift.[1]

2.2.1 Structural Analysis of Existing Agents

- **R-AGENT-01 (Architect):** Contains broad, static lists of principles ("Modularity," "Scalability") and process steps ("Requirements Gathering"). These are textbook definitions, not operational protocols specific to Stela. It essentially "lectures" the AI on what an architect is, rather than defining this specific architect's boundaries within the Stela repo.[1]
- **R-AGENT-02 (Code Reviewer):** Explicitly lists "Security Checks" and "Code Quality" rules. While valuable, these rules are hard-coded in the agent file. If the project updates its security standards (e.g., switching from JWT to PASETO), this agent file becomes stale ("Drift").
- **Duplicated Governance:** Crucially, these files implicitly or explicitly repeat behavioral rules. R-AGENT-02 instructs "Provide feedback organized by priority," which overlaps with the PoT's "Quantitative Reporting" standard.[1]

The failure mode here is clear: because these files are static and hand-written, they drift from the actual capabilities of the AI models and the evolving constraints of the repository. There is no proof that R-AGENT-01 has ever successfully architected anything in Stela; it is a theoretical construct.

2.2.2 The "Context Hazard" of Agents

Currently, the PoT explicitly lists docs/library/agents as a Context Hazard.

- **Negative Constraint:** "ops/lib/manifests/CONTEXT.md must exclude docs/library/agents, docs/library/tasks, and docs/library/skills".[1]
- **Current Friction:** Operators likely struggle with this because to use an agent, they must currently copy-paste the agent definition or manually load it, risking context contamination.
- **Solution Requirement:** The new system must allow for Just-In-Time (JIT) Injection of agents without violating the manifest rules. The Agent file must be "loadable" on demand via the Dispatch Packet.[1]

2.3 The "Pointer-First" Deficiency

The user query explicitly mandates a "Pointer-first agent constitution" with "no duplicated canon text." This aligns with Stela's llms.txt philosophy found in the dump.[1]

In many AI repositories, system prompts are monolithic blocks of text that repeat the same instructions (e.g., "You are a helpful assistant," "Do not use markdown in titles") across multiple files. Stela rejects this redundancy.

- **SSOT (Single Source of Truth):** Information must exist in exactly one place. All other references must be links or pointers to that canonical location.
- **Efficiency:** Repeating the behavioral logic standard (e.g., "No contractions") in every Agent file (R-AGENT-01 through 06) creates a maintenance nightmare. If the PoT changes to allow contractions, every agent file is instantly "Drifted" (out of date).
- **Implementation:** The new Agent files must strip all behavioral instructions and simply point to PoT.md Section 2.[1]

---

3. The Pointer-First Doctrine: A Constitutional Amendment

To resolve the asymmetry described above, Stela must adopt the "Pointer-First" doctrine for all agent definitions. This doctrine shifts the nature of an agent file from a "Textbook" (containing knowledge) to a "Router" (containing connections).

3.1 Theory of Pointer-First Architecture

The core tenet of Pointer-First architecture is that an Agent Definition should not contain *knowledge*; it should contain *access patterns*.

- **Content-First (Legacy):** An agent file explains "Filing Doctrine," "How to Verify," and "How to Speak."
  - *Risk:* Duplication, Drift, Hallucination.
- **Pointer-First (Target):** An agent file points to PoT.md Section 1.1, points to S-LEARN-01 for verification, and points to PoT.md Section 2 for speech patterns.
  - *Benefit:* Single Source of Truth, Lightweight Context, High Compliance.

This transforms the Agent from a repository of static text into a dynamic graph node that connects the AI to the specific "living" documents required for the task.

3.2 Refactoring the Agent Library

We must refactor docs/library/agents/ to align with this doctrine. The existing files (R-AGENT-01 through 06) must be purged of duplicated canon.

3.2.1 The New Agent Schema

The new R-AGENT-XX files will adhere to a strict schema designed to enforce the Pointer-First doctrine.[1]

Section	Content Strategy	Pointer Target (SSOT)	Rationale
Header	Name, Description, Model	N/A	Defines the persona identity.
Provenance	Immutable capture data	ops/lib/scripts/agent.sh	Proof of Work. Prevents hallucinated capabilities.
Constitution	"Follow these laws."	PoT.md	Removes behavioral duplication (e.g., linguistic precision).
Jurisdiction	"Where you can work."	docs/GOVERNANCE.md	Defines "Tone Lanes" and "Non-Negotiables."
Output	"How to report."	TASK.md	Enforces the "Receipt" and "Verification" formats.
Toolchain	"What you can run."	ops/bin/, tools/lint/	Explicitly authorizes binary usage (e.g., ops/bin/dump).
Skills	"What you know."	docs/library/skills/S-LEARN-XX	JIT-loads specific technical know-how.
Scope	"Seclusion	projects/ (Allowlist)	Defines the

	boundary."		"Seclusion" boundary for the agent.

3.2.2 Refactoring R-AGENT-02 (Code Reviewer)

As a case study, consider the refactoring of R-AGENT-02.

Current State (Legacy):

Code Reviewer

...

Behavioral Logic

- Linguistic Precision: Avoid contractions.
- No Direct Pushes: Do not attempt to push to main.

Security Checks

- Hardcoded credentials...
- SQL injection...

Target State (Pointer-First):

R-AGENT-02: Code Reviewer

Provenance

Captured: 2026-02-06... (Generated by agent.sh)

Constitution & Jurisdiction

- Constitution:] (Strict adherence required)
- Governance:]

Authorized Toolchain

- [[ops/bin/dump]]
- [[tools/lint/truth.sh]]

JIT Skills (Load on Demand)

-] (Security Protocols)

-] (Coding Standards)

By removing the explicit list of security checks and replacing it with a pointer to S-LEARN-05 (Security Protocols), we ensure that if the security protocols are updated, the Code Reviewer agent automatically inherits the new standards without requiring a file edit.

3.3 The Agent Registry (INDEX.md)

Currently, AGENTS.md is mentioned in the user query, but the file structure in the dump shows individual agent files. To align with the docs/library/ pattern, we must establish docs/library/agents/INDEX.md as the authoritative registry.[1]

Proposed Registry Format:

Agent Registry (Pointer-First)

ID	Name	Provenance (DP)	Specialization
R-AGENT-01	Architect	DP-OPS-0010	System Design, Trade-offs
R-AGENT-02	Code Reviewer	DP-OPS-0012	Security, Style, Perf

This registry serves as the entry point for the "Seclusion" logic, allowing the system to verify that a requested agent ID is valid and active.

---

4. Operational Specification: The Agent Provenance Protocol

To operationalize the new Agent System, we must build the tooling to support the "Harvest and Promote" lifecycle. This involves creating a new binary script, ops/lib/scripts/agent.sh, modeled after the successful skill.sh.[1]

4.1 The Harvest Engine (agent.sh harvest)

The harvest command is responsible for capturing the ephemeral state of a successful agent

session and crystallizing it into a draft artifact.

Command:

./ops/lib/scripts/agent.sh harvest --name "Dump Refactor Specialist" --dp "DP-OPS-0029"

Logic Flow:

1. **Context Capture:** Scrape the current OPEN prompt and loaded context files.
2. **Git Forensics:** Capture HEAD, Branch, and Friction Context (Hot Zone/High Churn) using ops/lib/scripts/heuristics.sh.
3. **Draft Generation:** Create a file in storage/handoff/ using the new Pointer-First Schema.
4. **Provenance Injection:** Insert the immutable YAML block proving the origin of the agent.

Output:

storage/handoff/agent-draft-YYYYMMDD-slug.md

4.2 The Promotion Gate (agent.sh promote)

The promote command is the quality control gate. It transforms a draft into a canon artifact. Crucially, it must enforce the "Pointer-First" doctrine via automated linting.[1]

Command:

./ops/lib/scripts/agent.sh promote <draft_path>

Logic Flow:

1. **Context Hazard Check:** Ensure the draft does not attempt to hard-code lists of skills or other library artifacts that would violate the Manifest.
2. **PoT Duplication Linter (CRITICAL):**
   - The script must scan the draft for forbidden phrases that indicate duplicated governance.
   - *Forbidden:* "No contractions", "Policy of Truth", "Do not edit main", "Linguistic Precision".
   - *Action:* If found, FAIL the promotion with an error: "Agent draft contains duplicated global governance. Remove text and use a pointer to PoT.md."
3. **Registry Action:**
   - Assign next available ID (R-AGENT-XX).
   - Move file to docs/library/agents/.
   - Update docs/library/agents/INDEX.md.
   - Update SoP.md (State of Play) to record the new capability.

4.3 Provenance Block Specification

The Provenance Block is the cryptographic chain of custody for the agent. It differentiates a "hallucinated" agent from a "verified" one.

Provenance

- **Captured:** 2026-02-06 14:00:00 UTC
- **DP-ID:** DP-OPS-0029
- **Branch:** work/agent-refactor
- **HEAD:** 83892f891
- **Friction Context:**
  - Hot Zone: docs/library/agents/
  - High Churn: ops/lib/scripts/

This block allows future Operators to audit *why* an agent exists. If an agent was harvested during a critical incident ("High Churn"), it may be more trusted for emergency work than one harvested during routine maintenance.

---

5. Seclusion Doctrine: The Future of Project Development

The user query mentions "SECLUSION TO BE EVENTUALLY USED ALONG WITH SKILLS AND TASKS." This points to the next evolution of Stela's architecture: Project Seclusion.[1]

5.1 The Problem with Global Context

Currently, the "Small Bundle" (llms-small.txt) and "Full Bundle" (llms-full.txt) are global. They load the platform's operating system (ops/, docs/).[1] However, as projects/ grow (e.g., projects/php-nuke-ce or new payloads), loading the global platform context alongside project-specific code risks two failures:

1. **Context Overflow:** The AI forgets platform rules because the project code is too large.
2. **Scope Leak:** The AI tries to apply platform governance rules (e.g., "Don't edit main") to a project that might use a different workflow (though Stela forbids this, the risk of confusion exists).

5.2 Defining Seclusion

Seclusion is the architectural state where an Agent operates within a hermetically sealed context containing *only* the artifacts necessary for the specific Dispatch Packet.

The Seclusion Equation:

$$SeclusionContext = PoT + DP + R_AGENT + JITSkills + TargetProjectFiles$$

Crucially, Seclusion excludes:

- Unrelated docs/ files.
- Unrelated ops/ scripts (unless in the toolchain).
- Other projects/.
- The rest of the library/.

5.3 Implementing Seclusion via ops/bin/project

The ops/bin/project binary [1] is the vehicle for Seclusion.

- **Current State:** It assembles OPEN + dump (platform) + STELA.md (project).
- **Target State:** It should assemble a **Secluded Bundle**:
  - **Base:** PoT.md, TASK.md, SoP.md.
  - **Agent:** The specific R-AGENT-XX requested.
  - **Project:** The specific projects/<id> files (via ops/bin/dump --scope=project).
  - **Skills:** Only the S-LEARN items cited in projects/<id>/STELA.md or the DP.

This prevents "Context Hazard" by default. The Agent literally cannot hallucinate about other parts of the system because they do not exist in its universe.

---

6. Implementation Roadmap and Dispatch Packet Analysis

The immediate next step is to execute the upgrade via **DP-OPS-0029**. This Dispatch Packet will operationalize the findings of this report.

6.1 DP-OPS-0029 Strategy

The DP (drafted as part of this analysis) targets the creation of the agent.sh script and the refactoring of the existing agents.

- **Objective:** Upgrade Agent System to Pointer-First Constitution and implement Provenance Lifecycle.
- **Target Files:**
  - ops/lib/scripts/agent.sh (New binary).
  - docs/library/agents/INDEX.md (New registry).
  - docs/library/agents/R-AGENT-01 through 06 (Refactor).
  - docs/library/INDEX.md (Update pointers).
  - SoP.md (Ledger update).

6.2 Execution Sequence

1. **Tooling First:** The Contractor must first create agent.sh by cloning and adapting skill.sh. This provides the machinery for the upgrade.
2. **Registry Creation:** Create docs/library/agents/INDEX.md.
3. **Refactor:** Systematically rewrite each R-AGENT file. This is a manual "harvest" process where the Contractor acts as the "refining fire," stripping away the legacy text and inserting the new pointers.
4. **Verification:** Run tools/lint/library.sh to ensure the new registry is valid and tools/lint/context.sh to ensure no context hazards were introduced.

6.3 Verification and Drift Control

The success of this upgrade will be measured by the **Drift Preventers** encoded in the new agents.

- **Negative Check:** If an agent file contains the string "No contractions," the upgrade has failed.
- **Positive Check:** Every agent must have a valid ## Provenance block.
- **Equilibrium:** The system reaches equilibrium only when all 6 agents are refactored, the registry is live, and SoP.md reflects the structural change.

6.4 Conclusion

The Stela repository's strength lies in its refusal to rely on implicit knowledge. By applying the Harvest and Promote pattern—proven in the Skills System—to the Agent System, Stela closes a critical governance gap. Agents will cease to be static, potentially drifting definitions and will become provenance-backed operational artifacts. This shift enables the eventual implementation of Seclusion, where agents operate in high-hygiene, context-safe environments, interacting with projects/ without the risk of platform contamination. This is the fulfillment of the Pointer-First Constitution: a system where every instruction can be traced to a single source of truth, and every active agent is a verified product of the system's own history.

Works cited

1. Agent System Upgrade Plan.pdf